

React

Course 2025-2026



Link to [code examples](#) in these slides

Team



Candela García Cruz

candela.cruz.13@ull.edu.es



Sara Darías Sánchez

sara.darias19@ull.edu.es



Sergio de la Barrera García

sergio.barrera.garcia.25@ull.edu.es

Index

- Introduction to React
- React setup with Vite
- Components + TSX
- Hello World
- Events in React
- Props
- Hooks
- useState + useRef
- useMemo + useEffect
- MVC in React
- Canvas in React
- Interactive canvas with state
- Final demo
- Our thoughts
- Conclusions

Introduction to React

Why do frameworks exist?

Consider a to-do list creator:

- Render a list of tasks.
- Add a new task.
- Delete a task

Add to-do task	Add
☐ Go to the gym	Delete
☐ Clean living room	Delete
☐ Reply to emails	Delete

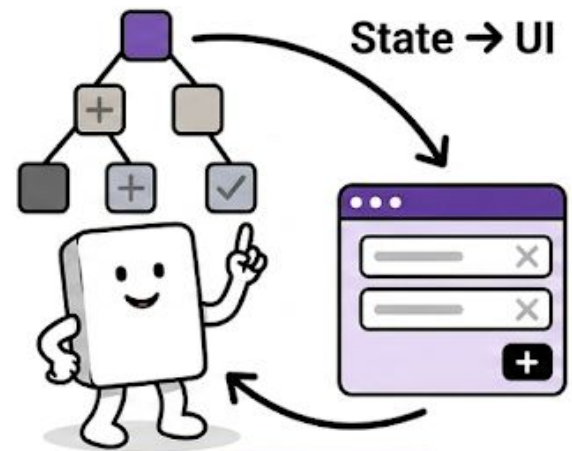
Why do frameworks exist? (II)

Consider a to-do list creator:

- Render a list of tasks.
- Add a new task.
- Delete a task

Each of our goals is theoretically simple in isolation.

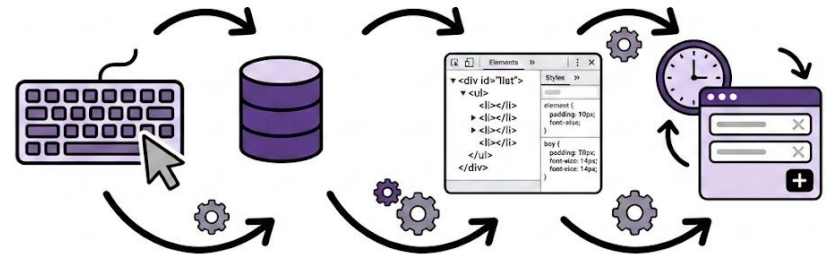
But... **every time we change our application's state, we need to update the user interface to match.**



Why do frameworks exist? (III)

Without a framework:

- Add a new task.
 1. Detect the user input.
 2. Update the data.
 3. Manually update the DOM.
 4. Refresh related elements (counter, list, etc.)



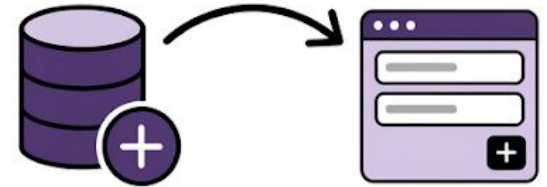
Every state change requires manual interface updates.

- More features (more code).
- More DOM manipulation.
- **More complexity and possible errors.**

Why do frameworks exist? (IIII)

With a framework:

- Add a new task.
 1. Update the state.
 2. The framework updates the DOM automatically.



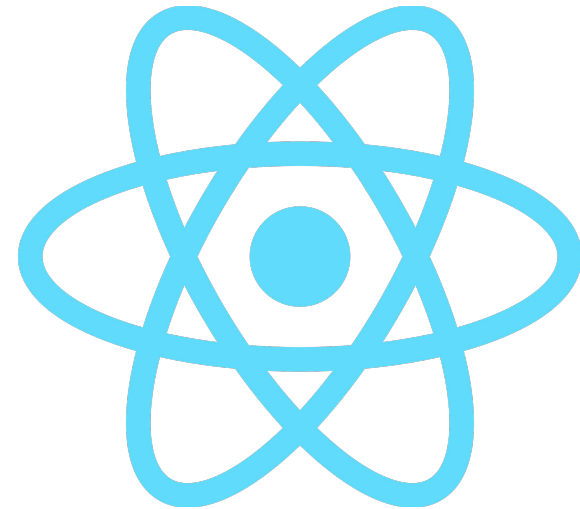
Frameworks provide a better developer experience.

They don't bring brand-new powers to TypeScript...

...they give you easier access to TypeScript powers so you can develop web applications more effectively in day-to-day work.

React

- Open-source JavaScript library developed by Facebook in 2011.
- Designed to create user interfaces.
- Based on reusable components.
- Follows a declarative approach.
- Uses a **virtual DOM** to efficiently update the user interface.
- Uses **TSX** syntax to write HTML-like code inside TypeScript (more readable).



Which framework is the best?

React (II)

NETFLIX



Uber
Eats



WordPress



TESLA

ebay

React setup

```
npm create vite@latest [your project name]
```

React setup (II)

```
> npx  
> create-vite trabajo-react
```

```
|  
◆ Select a framework:
```

- Vanilla
- Vue
- React
- Preact
- Lit
- Svelte
- Solid
- Qwik
- Angular
- Others

React setup (III)

```
> npx  
> create-vite trabajo-react
```

```
|  
◇ Select a framework:  
  React
```

```
◆ Select a variant:  
  ● TypeScript  
  ○ TypeScript + SWC  
  ○ JavaScript  
  ○ JavaScript + SWC  
  ○ React Router v7 ↗  
  ○ TanStack Router ↗
```

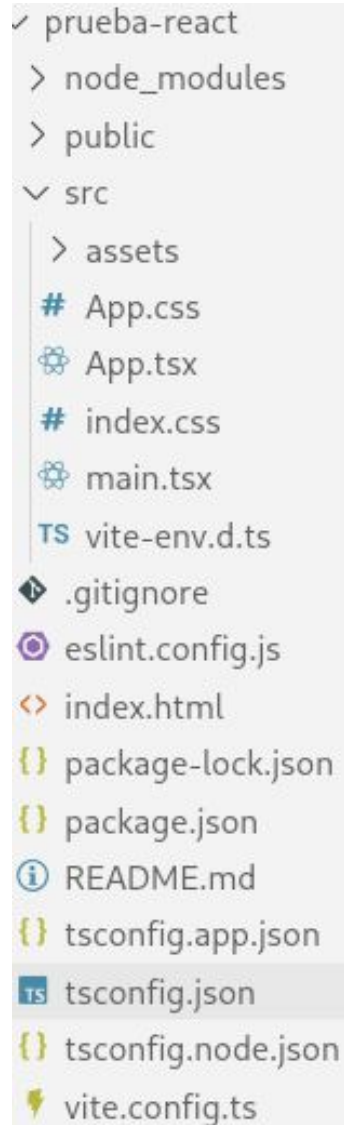
React setup (IV)

```
> npx  
> create-vite trabajo-react
```

```
|  
◇ Select a framework:  
  React  
|  
◇ Select a variant:  
  TypeScript  
|  
◇ Scaffolding project in /home/usuario/Trabajo/trabajo-react...  
└─ Done. Now run:
```

```
cd trabajo-react  
npm install  
npm run dev
```

React setup (V)



A screenshot of a file explorer window showing the directory structure of a project named 'prueba-react'. The 'src' directory is expanded, showing subdirectories like 'assets' and files like 'App.css', 'App.tsx', 'index.css', 'main.tsx', and 'vite-env.d.ts'. Other files in the root include '.gitignore', 'eslint.config.js', 'index.html', 'package-lock.json', 'package.json', 'README.md', 'tsconfig.app.json', 'tsconfig.json' (highlighted), 'tsconfig.node.json', and 'vite.config.ts'.

- ✓ prueba-react
 - > node_modules
 - > public
 - ▼ src
 - > assets
 - # App.css
 - ⚙ App.tsx
 - # index.css
 - ⚙ main.tsx
 - TS vite-env.d.ts
 - 📄 .gitignore
 - 🔗 eslint.config.js
 - 📄 index.html
 - 📄 package-lock.json
 - 📄 package.json
 - 📄 README.md
 - 📄 tsconfig.app.json
 - TS tsconfig.json**
 - 📄 tsconfig.node.json
 - ⚡ vite.config.ts

JSX / TSX

What is JSX?

- JS + XML

Characteristics

- Expressions with curly braces
- One Top Level Element
 - Fragments `<></>`
- `className`
- Comments `{/* */}`
- `import .css`

[tsx-example.tsx](#)

React implementations

React structure

- React uses a container to render HTML in a web page.
- Typically, this container is a `<div id="root"></div>` element in the `index.html` file.



```
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <title>2025-2026-pai-react</title>
5      <script type="module" src="/src/main.tsx"></script>
6    </head>
7    <body>
8      <div id="root"></div>
9    </body>
10 </html>
```

[index.html](#)

React structure (II)

- *createRoot* is a built-in function used to create a root node for a React application.

Defines the HTML element where a React component should be displayed.

- The *render* method defines what to render in the HTML container.



```
1  createRoot(document.getElementById('root')).render(  
2    <StrictMode>  
3      <App />  
4    </StrictMode>,  
5  )
```

Components

What are components?

They are independent and reusable bits of code.



It is now suggested to use Function components along with Hooks, instead of Class components.

- The component's name **MUST** start with a capital letter.

```
1  function Greeting() {  
2    return (  
3      <h1>Hello, welcome to React</h1>  
4    );  
5  }
```

[components-example.tsx](#)

Hello world (II)

- Content that is displayed in the container:



```
1  import type { JSX } from 'react';
2
3  export function HelloWorld(): JSX.Element {
4    return (
5      <div className="App">
6        <h1>Hello World!</h1>
7      </div>
8    );
9  }
```

[hello-world.tsx](#)

Events in React

Event handlers

What are Handlers?

your own functions that will be triggered in response to interactions

How to use them?

1. Define a function
2. Pass it as a prop to the appropriate JSX tag

Events in React (II)

Ways to define a handler

Inline



```
1 function Button() {  
2   return (  
3     <button onClick={() =>  
4       alert('Button clicked!')}>  
5       Click me  
6     </button>  
7   );  
8 }
```

[events.tsx](#)

Normal function



```
1 function Button() {  
2   function handleHover() {  
3     console.log('Button  
4       hovered!');  
5   }  
6  
7   return (  
8     <button  
9       onClick={handleHover}>  
10      Click me  
11    </button>  
12  );  
13 }
```

Props

- Props are arguments passed into React components.
- Props are passed to components via HTML attributes.



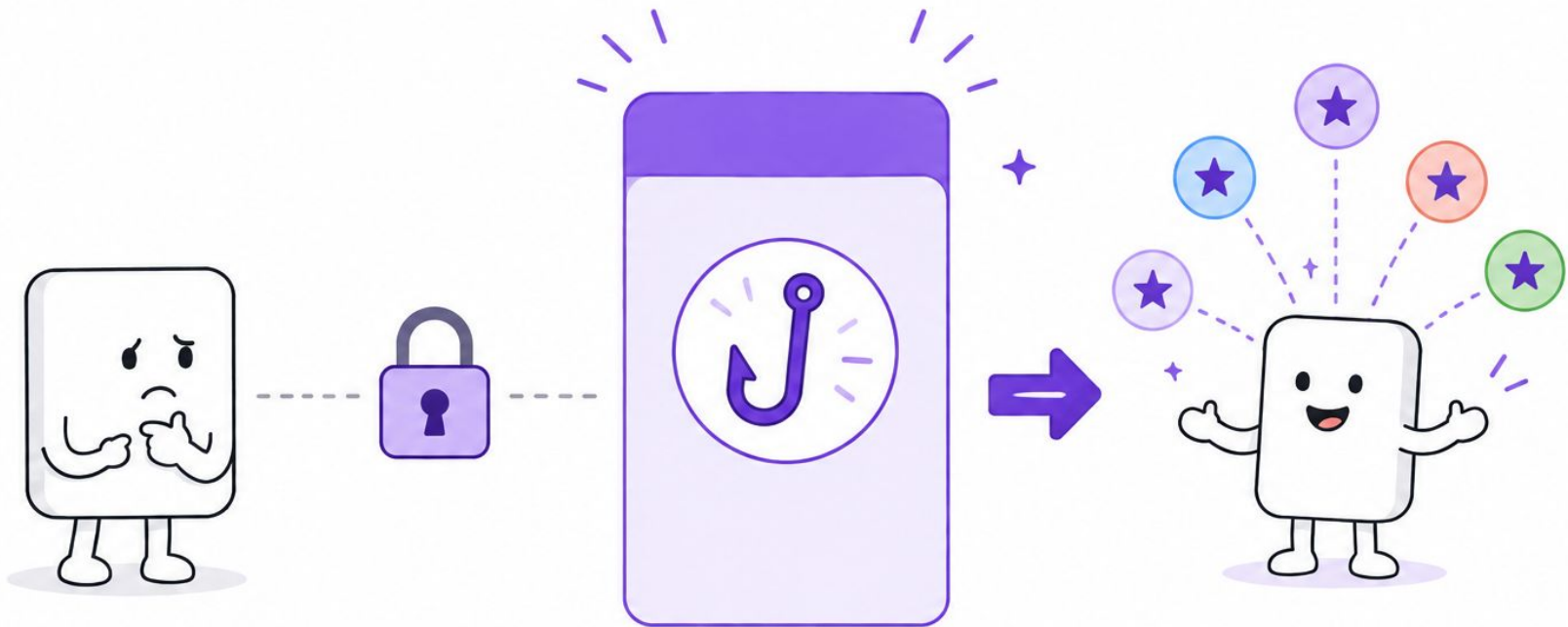
```
1  type BookProps = {  
2    title: string;  
3    author?: string;  
4    details?: {  
5      year: number;  
6      genre: string;  
7    };  
8    tags?: string[];  
9    extra?: string;  
10 };
```

[book-props.tsx](#)

Hooks

What are Hooks?

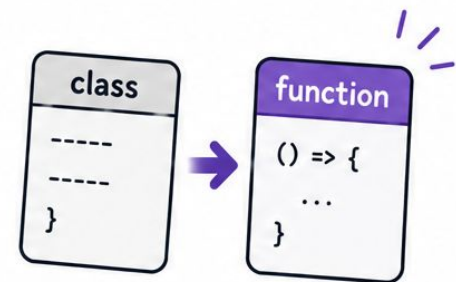
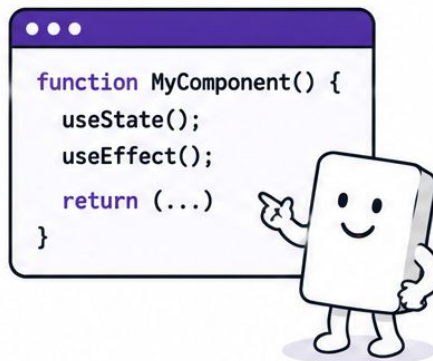
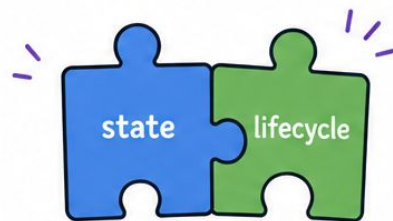
They are special React functions that allow functional components to utilize advanced capabilities



Hooks (II)

What capabilities are these?

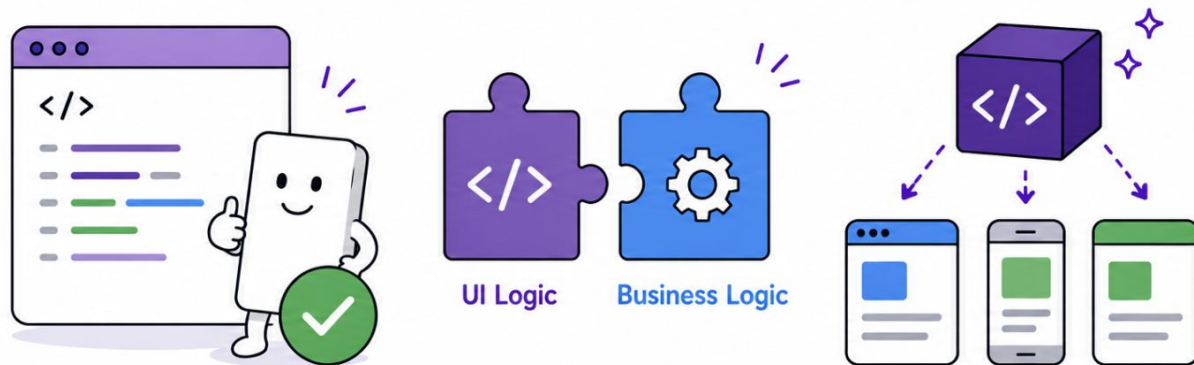
- Let you use state and lifecycle
- Work inside functional components
- Replace class-based components



Hooks (III)

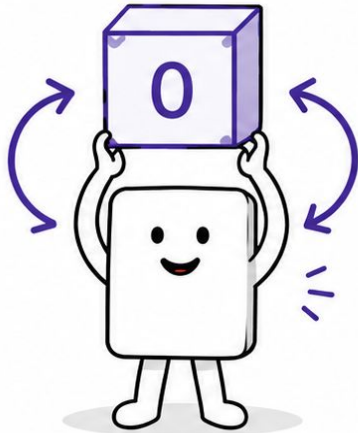
Why use Hooks?

- Cleaner and simpler code
- Better separation of logic
- Easier to reuse logic

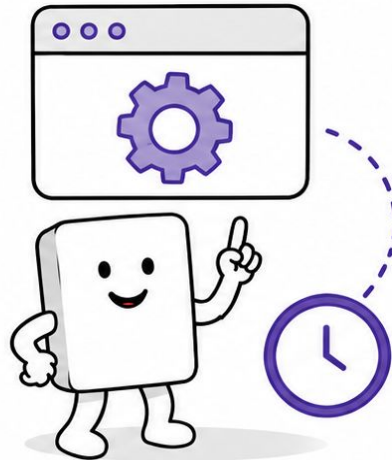


Hooks (IV)

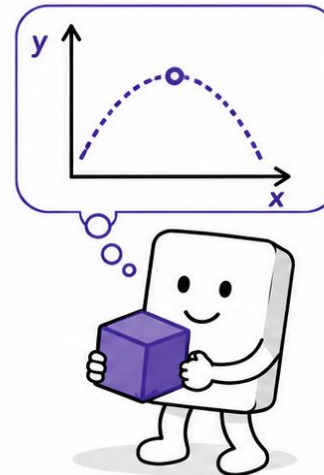
Most used Hooks



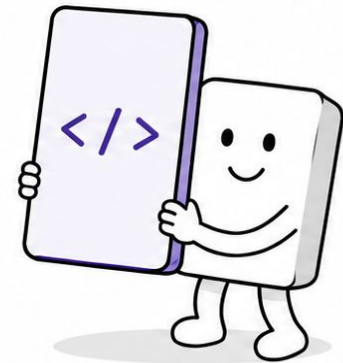
useState



useEffect



useMemo



useRef



React provides **20+** built-in Hooks to cover a wide range of needs.

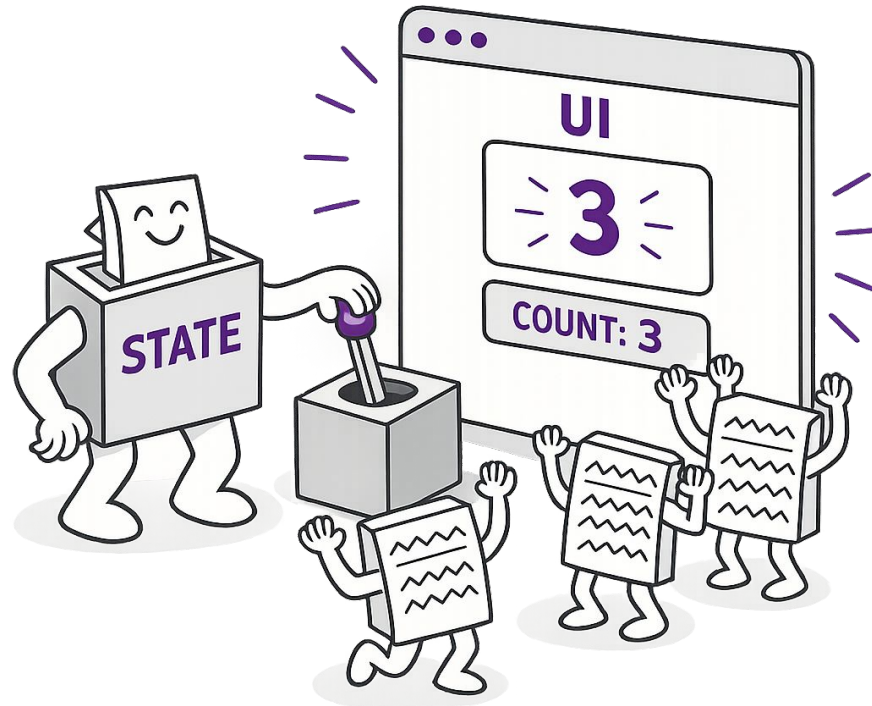


react.dev/hooks

useState

State with useState

- Dynamic data
- Updates the UI automatically



useState (II)

useState Hook

- value + updater function



```
1  const [count, setCount] = useState<number>(0);
```

useState (III)

Interactive Example

- Event → state change → UI update

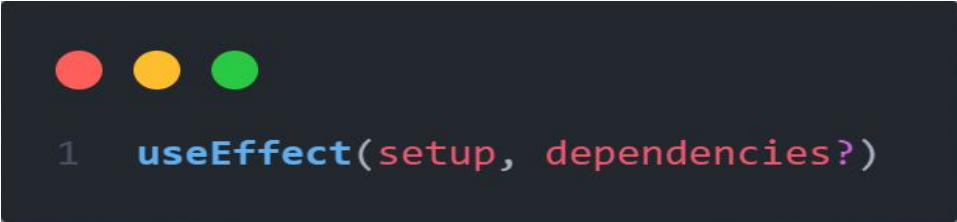


```
1 <button onClick={() => setCount(count + 1)}>  
2   Increment  
3 </button>
```

[use-state.tsx](#)

UseEffect

Lets you synchronize a component with an external system.



```
1  useEffect(setup, dependencies?)
```

two arguments

1. A setup function with setup code that connects to that system. It should return a cleanup function with cleanup code that disconnects from that system.
2. A list of dependencies including every value from your component used inside of those functions.

UseEffect (II)

Example



```
1  useEffect(() => {  
2    if (count % 2 === 0) {  
3      setColor('blue');  
4    } else {  
5      setColor('red');  
6    }  
7  }, [count]);
```

[use-effect.tsx](#)

useMemo

What does it do?

- Memoizes a calculation
- Avoids recalculating on every render
- Only runs when dependencies change



```
1  const trajectory = useMemo(() => {  
2    return calculateTrajectory(parameters);  
3  }, [parameters]);
```

UseRef

Lets you reference a value that's not needed for rendering.



```
1  const ref = useRef(initialValue)
```

Parameters

`initialValue`: The value you want the ref object's current property to be initially.

Returns

a ref object with a single current property initially set to the initial value you provided.

UseRef (II)

Example



```
1  const previousInputValue = useRef('');
```

[use-ref.tsx](#)

MVC in React

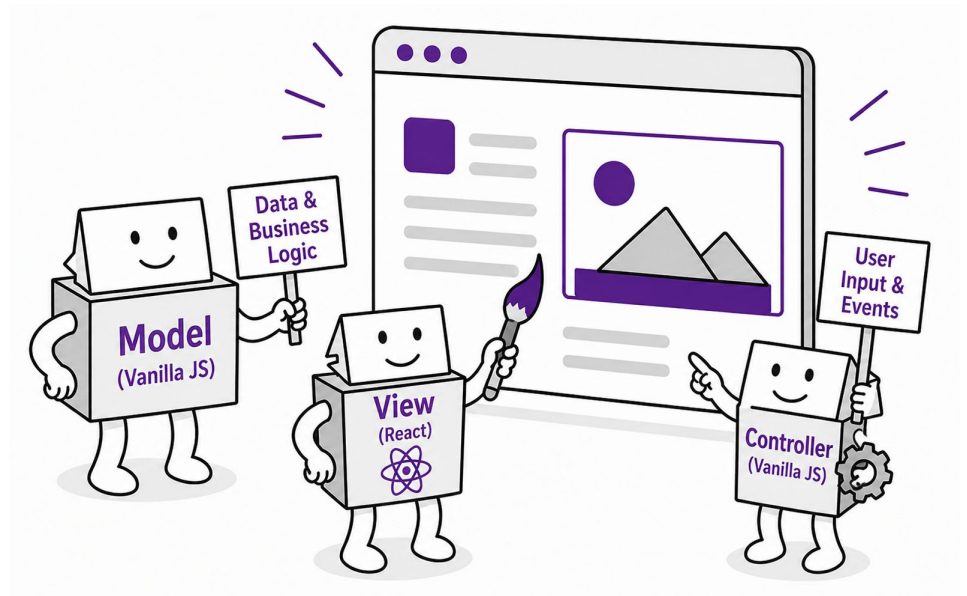
Model <- Vanilla

View <- React

Controller <- Vanilla

We need to have one component with everything

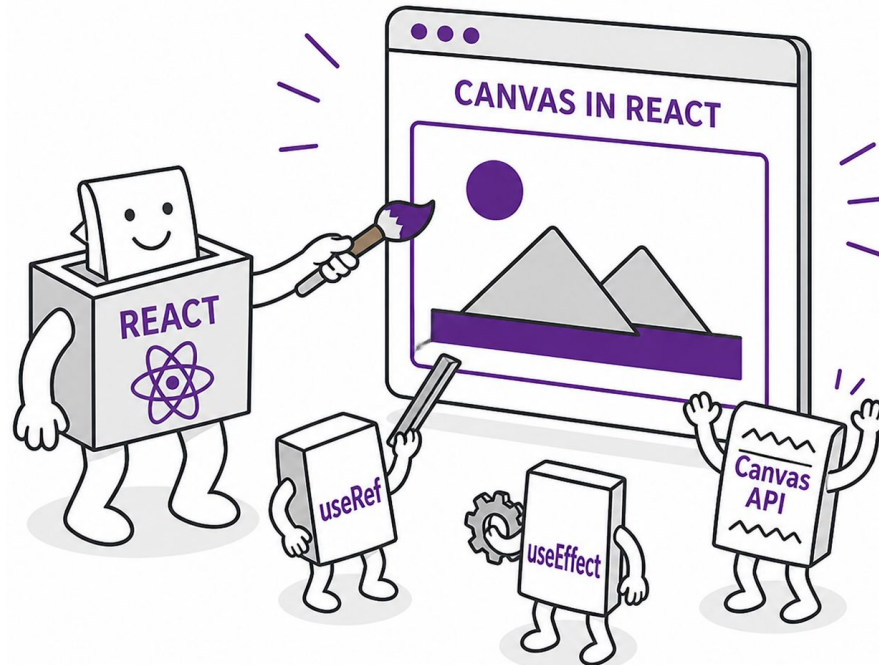
[MVC-example](#)



Canvas in React

Canvas remains the same

- HTML element for drawing graphics
- Useful for visual applications
- Drawing is done with React



Canvas in React (II)

Accessing Canvas

- useRef gives access to the canvas element



```
1  const canvasRef = useRef<HTMLCanvasElement | null>(null);
```

Canvas in React (III)

Drawing Canvas

- `useEffect` runs the drawing code



```
1  useEffect(() : void => {  
2    const canvas = canvasRef.current;  
3    const context = canvas.getContext('2d');  
4  }
```

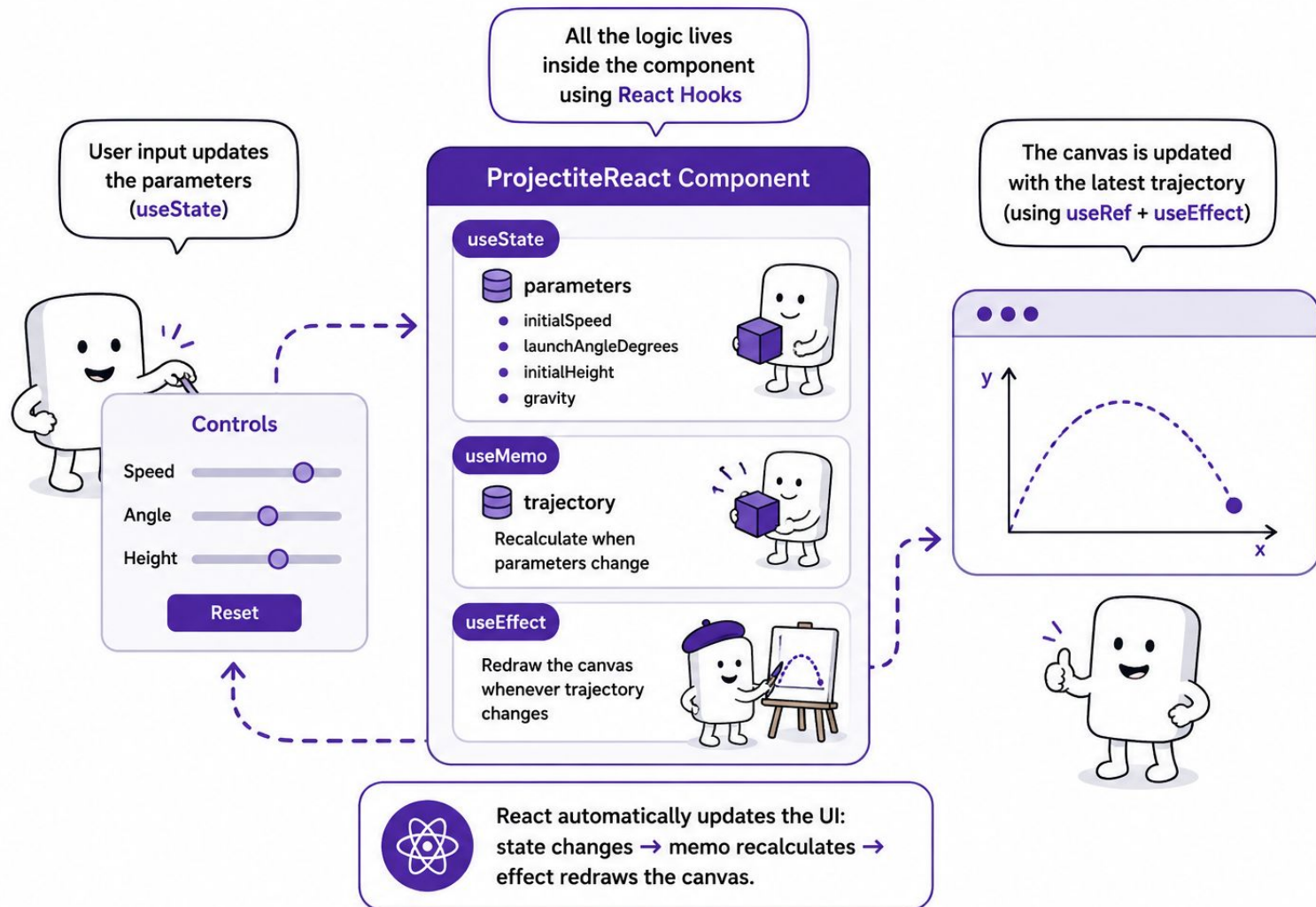
[canvas-with-react.tsx](#)

Final demo (Projectile)

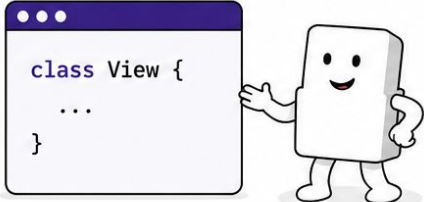
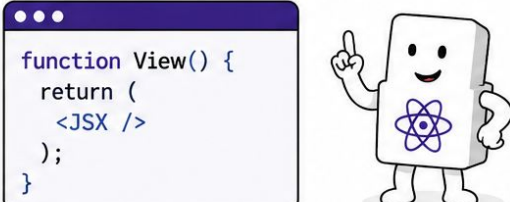
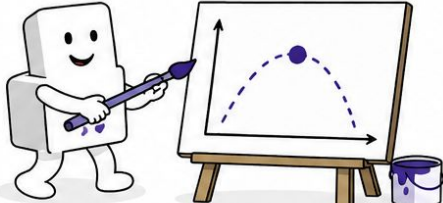
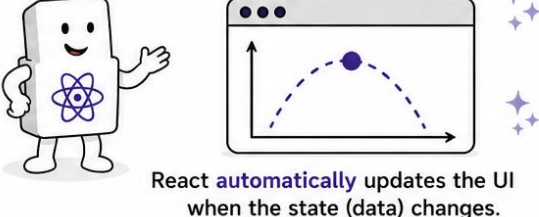
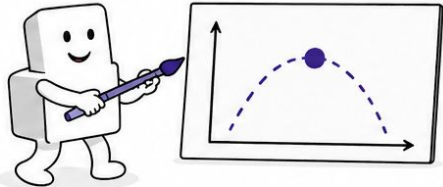
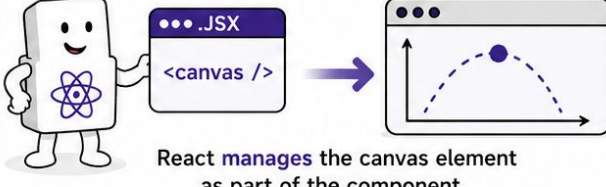
What we used?

- React Hooks
- State management
- Canvas rendering
- Controlled inputs
- Physics calculations
- Reactive UI updates

Final demo (Projectile II)

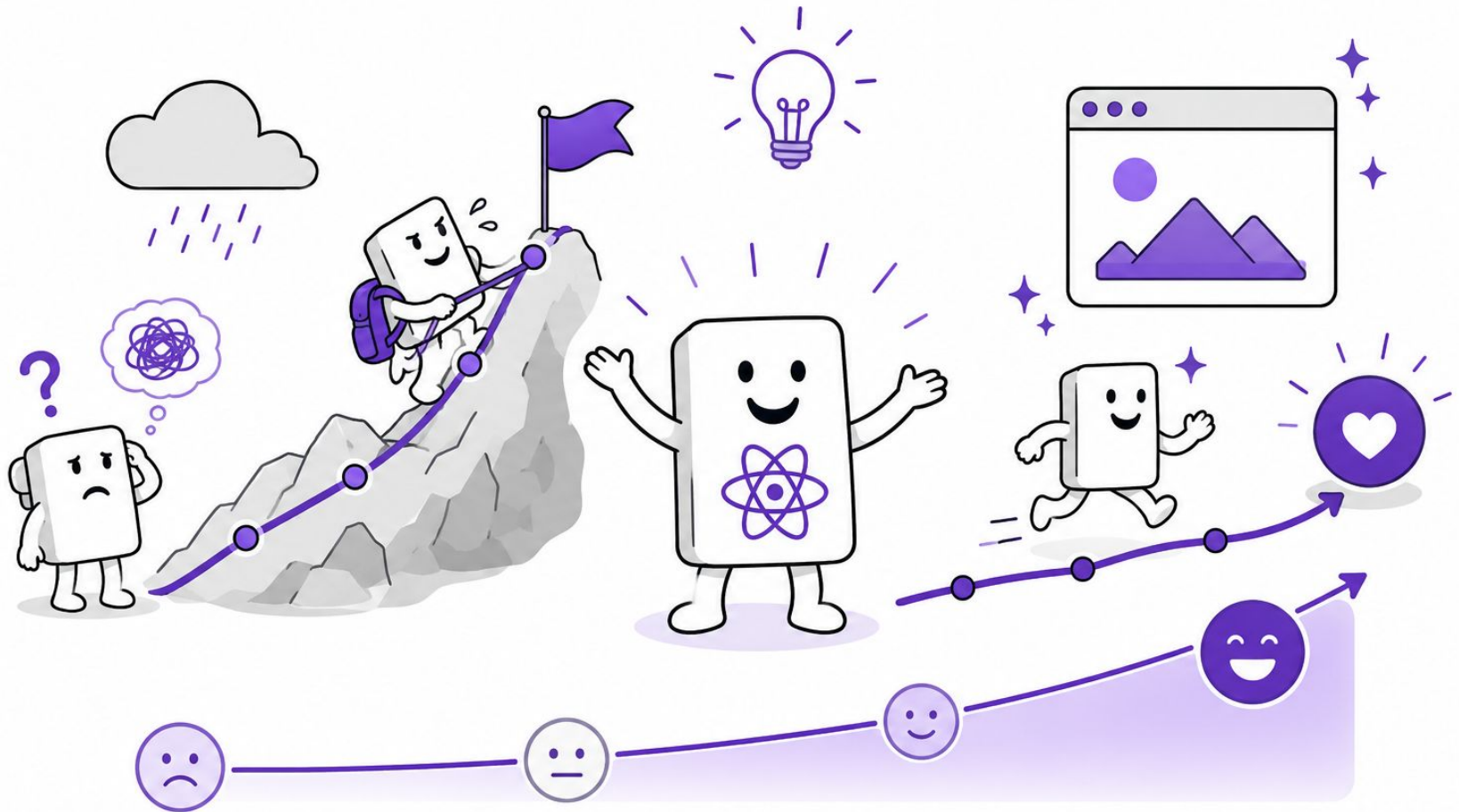


Final demo (Projectile III)

PAI (Traditional MVC)	React
View = class  <pre>class View { ... }</pre>	View = function component  <pre>function View() { return (<JSX />); }</pre>
Manual rendering 	Declarative rendering  React automatically updates the UI when the state (data) changes.
Direct canvas control 	Canvas inside React  React manages the canvas element as part of the component.

[projectile-react.tsx](#)

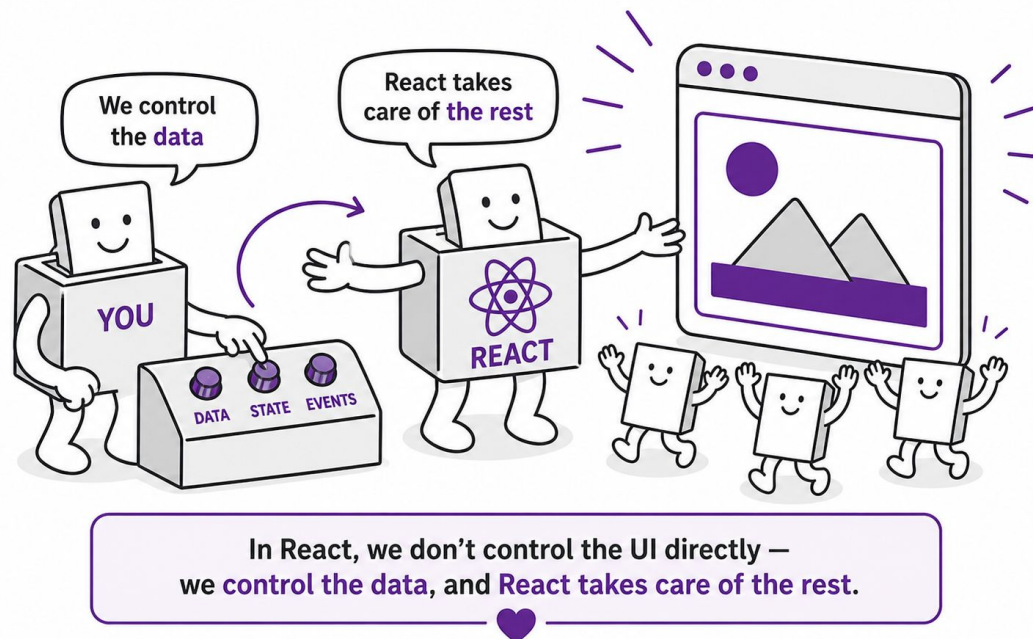
Our thoughts



Conclusions

React Summary

- Components → structure the UI
- State → controls dynamic data
- Events → handle user interaction
- Canvas → enables custom graphics



Bibliography

- [Introduction to client-side frameworks - Learn web development | MDN](#)
- [React - wikipedia](#)
- [Descripción General de la Referencia de React](#)
- [Why use React?](#)
- [Build a React app from Scratch](#)
- [**React Tutorial**](#)
- [**Midudev´s Tutorial**](#)
- [Responding to Events – React](#)
- [State: A Component's Memory – React](#)
- [Passing Props to a Component - React](#)
- [Creating an Interactive Canvas Component](#)
- [Canvas with React.js](#)

Bibliography (II)

- [JSX](#)
- [TSX](#)
- [useRef - React](#)
- [useEffect-React](#)
- [useState – React](#)
- [useMemo – React](#)

Any questions?